

Fissata una MT M , la sua **COMPLESSITA' TEMPORALE** (nel **CASO PESSIMO**) e' la funzione $T: \mathbb{N} \rightarrow \mathbb{N}$ t.c. $n \mapsto$ massimo numero di transizioni richieste per accettare o rifiutare un input di lunghezza n

In realta', si puo' parlare anche di **CASO OTTIMO** e **CASO MEDIO**, ma tipicamente, quello che ci interessa e' quanto puo' spingersi al limite un algoritmo ("quanto e' il massimo tempo impiegabile dalla MT per risolvere il problema?")

In realta', per rispondere a questa domanda possiamo fare delle semplificazioni e ragionare in termini "asintotici"

Se f e g sono due funzioni da $\mathbb{N}$ in $\mathbb{R}^+$, dico che **$f = O(g)$** se $\exists c \in \mathbb{R}^+$ t.c.
 $\exists n_0 \in \mathbb{N}$ t.c. $\forall n \geq n_0 \quad f(n) \leq c \cdot g(n)$

ESEMPIO: $n+100 = O(n^2)$, perche' a un certo punto risultera' che la funzione n^2 sara' maggiore di $n+100$ e ci rimarra' tale (dopotutto n^2 cresce piu' velocemente di $n \dots$)

NOTA: Se esiste $\lim_{n \rightarrow +\infty} \frac{g(n)}{f(n)} = l$, allora **$f = O(g)$** se $l > 0$

ESEMPIO: $\{x\} = x - \lfloor x \rfloor$ parte frazionaria di x

Allora $\{x\} = O(1)$, perche' la parte frazionaria non puo' essere mai maggiore di 1!

~~$f = \Theta(g)$~~ $\Leftrightarrow \exists c_1, c_2 > 0$ t.c. $\exists n_0 \in \mathbb{N}$ t.c.
 $\forall n \geq n_0 \quad c_1 g(n) \leq f(n) \leq c_2 g(n)$

~~$f = \Omega(g)$~~ $\Leftrightarrow \exists c > 0$ t.c. $\exists n_0 \in \mathbb{N}$ t.c.
 $\forall n \geq n_0 \quad f(n) \geq c g(n)$

① Confrontare ...

① $n!$ e e^n $\rightarrow$ cioè: $n! = O(e^n)$? Oppure,
 $n! = \Theta(e^n)$? Oppure ...

② e^n e $n \cdot 2^n$

③ $\log_2 n$ e $\log_{10} n$

④ $\log(n!)$ e $n \log n$

SOLUZIONE:

① Provo che $e^n = O(n!)$ e che $e^n \neq \Theta(n!)$...

$$\lim_{n \rightarrow +\infty} \frac{n!}{e^n} \geq \lim_{n \rightarrow +\infty} \frac{\left(\frac{n}{2}\right)^{n/2}}{e^n} = \lim_{n \rightarrow +\infty} \frac{e^{\frac{n}{2} \log(\frac{n}{2})}}{e^n} =$$
$$= \lim_{n \rightarrow +\infty} e^{\frac{n}{2} \log(\frac{n}{2}) - n} = \lim_{n \rightarrow +\infty} e^{n \left(\frac{1}{2} \log(\frac{n}{2}) - 1\right)} = +\infty$$

Quindi $e^n = O(n!)$. Poiché, poi, il limite non è
finito, vale che $e^n \neq \Theta(n!)$

② Dimostriamo che $n \cdot 2^n = O(e^n)$

$$\frac{e^n}{n \cdot 2^n} = \frac{e^n}{e^{\log n + n \log 2}} = e^{n - \log n - n \log 2} =$$

$$= e^{n(1-\log 2)} - \log n \xrightarrow{n \rightarrow +\infty} +\infty$$

In generale, si dimostra che ...

$$2^n = O((2,1)^n) \quad n^{100} 2^n = O((2,1)^n)$$

③ Notiamo che $\log_2 n = \frac{\log n}{\log 2}$ e $\log_{10} n = \frac{\log n}{\log 10}$

Quindi, il limite del loro rapporto è una costante $\rightarrow \log_2 n = \Theta(\log_{10} n)$

Questo vale qualsiasi sia la base del logaritmo!

④ $n \log n$ è abbastanza piccolo...

$$\forall \varepsilon > 0 : n \log n = O(n^{1+\varepsilon})$$

Cioè: $n \log n$ cresce di più di una lineare, ma meno di una quadratica...

$\log n!$ è molto vicino a $n \log n$. Infatti...

$$\frac{n}{2} \log \frac{n}{2} = \log \left(\left(\frac{n}{2} \right)^{\frac{n}{2}} \right) \leq \log(n!) \leq \log(n^n) = n \log n$$

$$\text{Quindi } \log(n!) = \Theta(n \log(n))$$

Questo tipo di complessità è molto comune per molti algoritmi (per esempio quelli di sorting...)

② Sia $L = \{ w \in \{a, b\}^* : \#_a(w) = \#_b(w) \}$

Descrivere una macchina astratta che riconosca L con la minima complessità spaziale* possibile

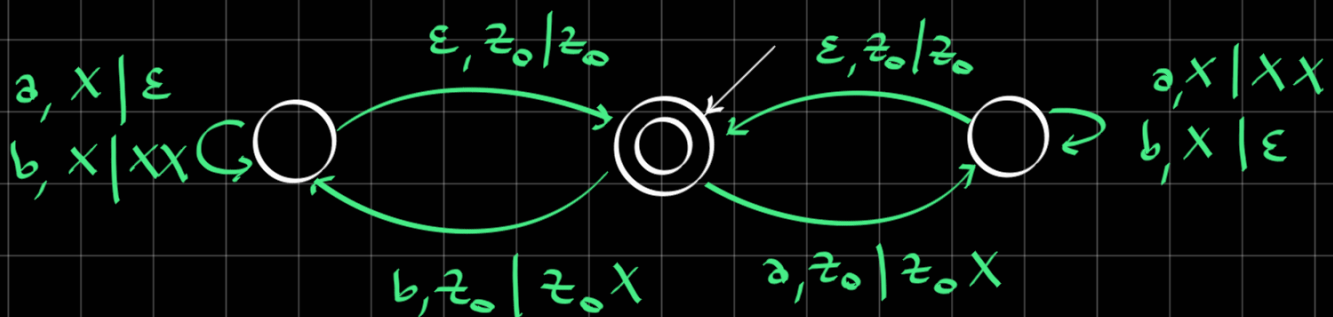
* Quindi ci interessa il valore asintotico, non il numero preciso di celle occupate

SOLUZIONE:

Dobbiamo scegliere tra:

- FSA $\longrightarrow$ complessità spaziale definita dal # di
- AP stati nel caso degli FSA
- MT

Sappiamo che l'accettore a potenza minima per L è un automa a pila...



Ma avere potenza minima non significa

necessariamente avere comp. spaziale minima

Se fissiamo una lunghezza $2n$. Allora...

- $(ab)^n$ utilizza una sola cella della pila. Questo è il **CASO OTTIMALE**
- a^{2n} utilizza $2n$ celle della pila. Questo è il **CASO PESSIMO**

RICORDA: **il caso pessimo non deve essere per forza una stringa accettata!**

Infatti, in questo caso, il caso pessimo è 2^{2n}

Per questo motivo, diciamo che la complessità spaziale è una funzione $S(n)$ con $S(n) = O(n)$

DOMANDA: Possiamo diminuire $O(n)$?

Possiamo immaginare che gli AP non possano fare di meglio. Generalmente, solo cambiando il modello di calcolo si possono fare grossi miglioramenti.

Proprio per questo, proviamo a usare una MT, ma a quanti nastri?

SICURAMENTE NON A NASTRO SINGOLO (useremmo un numero di celle pari alla lunghezza dell'input, quindi caso ottimale già pari a $2n$ (in realtà anche quello peggio vale $2n$))

RICORDA:

- MT nastro singolo: conto tutte le celle occupate (anche input e output)
- MT a k nastri: conto tutte le celle dei k nastri di memoria (non considero quelli di input e output!)

Usiamo due nastri in questo modo:

- Sul primo memorizzo $\#_a(w)$ in base 2
- Sul secondo memorizzo $\#_b(w)$ in base 2
- Confronto il contenuto dei due nastri e verifico che siano uguali!

va bene anche scrivere $\log_2(\#_{...}(w))$,

SPAZIO OCCUPATO:

$$\text{NASTRO 1} = \lceil \log_2(\#_a(w)) \rceil + 1$$

$$\text{NASTRO 2} = \lceil \log_2(\#_b(w)) \rceil + 1$$

→ tanto a noi interessa il comportamento asintotico

$$\text{Notiamo che } \text{NASTRO 1} + \text{NASTRO 2} \leq 2 \cdot \log_2(|w|)$$

Dunque, se la lunghezza dell'input è n , allora

$$S(n) = O(\ln(n))$$

$$2\log_2(n) \sim \log_2(n) = \frac{\ln(n)}{\ln(2)} \sim \ln(n)$$

Ripeto: a noi interessa il comportamento asintotico

→ le approssimazioni vanno bene.

Se vogliamo essere più precisi allora si può

$$\text{scrivere } S(n) = O\left(\max_{a+b=n} (\ln(a) + \ln(b))\right)$$

Problema di ottimizzazione vincolata

(X ESERCIZIO: Disegna la MT di questo algoritmo)

Questa MT ha una migliore complessità spaziale.

Ma dobbiamo ricordare che **generalmente, migliorare**

$S(n)$ peggiora $T(n)$ (e viceversa)

- L'AP di prima esegue una sola scansione dell'input SEMPRE $\rightarrow T(n) = \Theta(n)$

GIUSTO PER CAPIRE L'OSSERVAZIONE FATTA

- Questa MT fa due cose: conta in base due su due mastri e fa un confronto dei due risultati. Alla $i+1$ -esima iterazione del ciclo di conteggio sul primo mastro, esso occuperà $\log_2(i)$ celle e per passare a $i+1$ in base 2 dovrà modificare tutto il mastro (questo succede quando tutte le celle contengono '1'). Questo impiegherebbe fino a $O(\log(i))$ mosse. In totale, se l'input fosse 2^n , il tempo totale sarebbe $O\left(\sum_{i=1}^n \log(i)\right) = O\left(\log\left(\prod_{i=1}^n i\right)\right) = O(\log(n!))$

Proprietà dei logaritmi

$$n! \leq n^n$$

$$= O(n \log n)$$

- $\rightarrow$ le performance sono (apparentemente) peggiorate. Ma notiamo di aver fatto una supposizione MOLTO pessimistica: abbiamo ipotizzato che a ogni iterazione dobbiamo modificare tutte le celle del mastro, quando la metà delle volte non aggiorniamo niente (dunque il costo del ciclo di conteggio in questi casi sarebbe 3...), in $1/4$ dei casi aggiorniamo solo la prima cella (costo 5), in $1/8$ dei casi aggiorniamo solo la prima e seconda cella (costo 7), nel caso $n/16$... (costo 11)

Quindi, in generale, in $n/2^k$ casi aggiorniamo solo le prime k celle, con costo $3 + 2^{k-1}$

$$\text{TOTALE: } \sum_{i=1}^{\log(n)} \frac{1}{2^i} (3 + 2^{i-1}) = O(n)$$

RIASSUMENDO:

- Un ASF ha $\begin{cases} S(n) = \Theta(1) \rightarrow \text{numero di stati costante} \\ T(n) = \Theta(n) \rightarrow \text{leggo una sola volta tutta la stringa} \end{cases}$
 Θ perché questo vale sempre, per qualsiasi FSA

• Un AP (deterministico) ha...

$$\begin{cases} S(n) = O(n) \\ T(n) = \Theta(n) \rightarrow \text{come gli ASF} \end{cases}$$

Al massimo la complessità è lineare. Ma ci sono certi linguaggi che possono essere riconosciuti con complessità $\Theta(1)$ $\rightarrow$ Mettiamo $O(n)$ perché in certi casi si può scendere più in basso!

• Una MT a k nastri ha $\begin{cases} T(n) = \Omega(1) \\ S(n) = \Omega(1) \end{cases}$

Diciamo che questo non ci rappresenta niente: sicuramente più di una mossa la fa, ma non c'è modo di definirla meglio...

• Una MT a master singolo Re $\begin{cases} T(n) = \Omega(1) \\ S(n) = \Omega(n) \end{cases}$

Analogo a prima

Almeno la
lunghezza dell'input
devo accettarla!
